AWS VPC PEERING

Project Summary

This project demonstrates how to establish private network communication between two Amazon Virtual Private Clouds using VPC Peering, route tables, security groups, and private EC2 instances.

1. Project Overview

1.1 Introduction

Amazon Virtual Private Cloud (Amazon VPC) allows users to create logically isolated virtual networks in AWS. A VPC can contain public and private subnets, route tables, security groups, and EC2 instances.

AWS VPC Peering provides a networking connection between two VPCs, allowing resources to communicate using private IP addresses when the necessary routes and security rules are configured.

In this project, two VPCs were configured in the Mumbai AWS Region. A peering connection was established between them, and private EC2 instances were used to test network connectivity.

1.2 Project Objectives

The objectives of this project are:

1. Create two VPCs with non-overlapping CIDR ranges.
2. Create public and private subnets as required by the architecture.
3. Configure an Internet Gateway and route tables.
4. Establish an active VPC peering connection.
5. Configure forward and return routes between the VPCs.
6. Restrict inbound traffic using security groups.
7. Use a bastion host to access private EC2 instances.
8. Test connectivity before and after peering.
9. Perform a controlled failure test and verify recovery.
10. Document the configuration and results using screenshots.

1.3 Tools and Technologies

- Amazon Web Services (AWS)
- Amazon VPC
- VPC Peering
- Amazon EC2
- Internet Gateway
- Route Tables
- Security Groups
- Amazon Linux 2023
- OpenSSH
- Linux terminal commands
- Microsoft Word and PDF
- GitHub for project documentation

2. Network Architecture and Resource Inventory

2.1 Architecture Overview

The project contains two VPCs connected through a VPC peering connection.

VPC-A

- VPC name: ewb-vpc-a
- CIDR: 10.10.0.0/16
- Public subnet: 10.10.1.0/24
- Private subnet: 10.10.2.0/24
- Bastion host: ewb-bastion
- Private application server: ewb-app-a

VPC-B

- VPC name: ewb-vpc-b
- CIDR: 10.20.0.0/16
- Private subnet: 10.20.1.0/24
- Private application server: ewb-app-b

2.2 Resource Inventory

Resource	Name	Address or Identifier
VPC-A	ewb-vpc-a	10.10.0.0/16
VPC-B	ewb-vpc-b	10.20.0.0/16
VPC-A public subnet	ewb-a-public	10.10.1.0/24
VPC-A private subnet	ewb-a-private	10.10.2.0/24
VPC-B private subnet	ewb-b-private	10.20.1.0/24
Bastion	ewb-bastion	Record actual private and public IPs
App-A	ewb-app-a	10.10.2.10
App-B	ewb-app-b	10.20.1.10
Peering connection	ewb-a-b-peer	pcx-00e11115874e94503

2.3 Actual AWS Resource IDs

Record the IDs shown in your AWS Management Console.

- VPC-A ID: vpc-022910da01aa29aa7
- VPC-B ID: vpc-02436817e02d07041
- VPC-A public subnet ID: subnet-03ba07aa3d78220dc
- VPC-A private subnet ID: subnet-04961c9176d9a789f
- VPC-B private subnet ID: subnet-09f94181ad141a26d

- Peering connection ID: pcx-00e11115874e94503
- App-A instance ID: i-0cb67d35a5edfea09
- App-B instance ID: i-069e1764a2e225a25
- Bastion instance ID: i-02ef4c50f90263a56

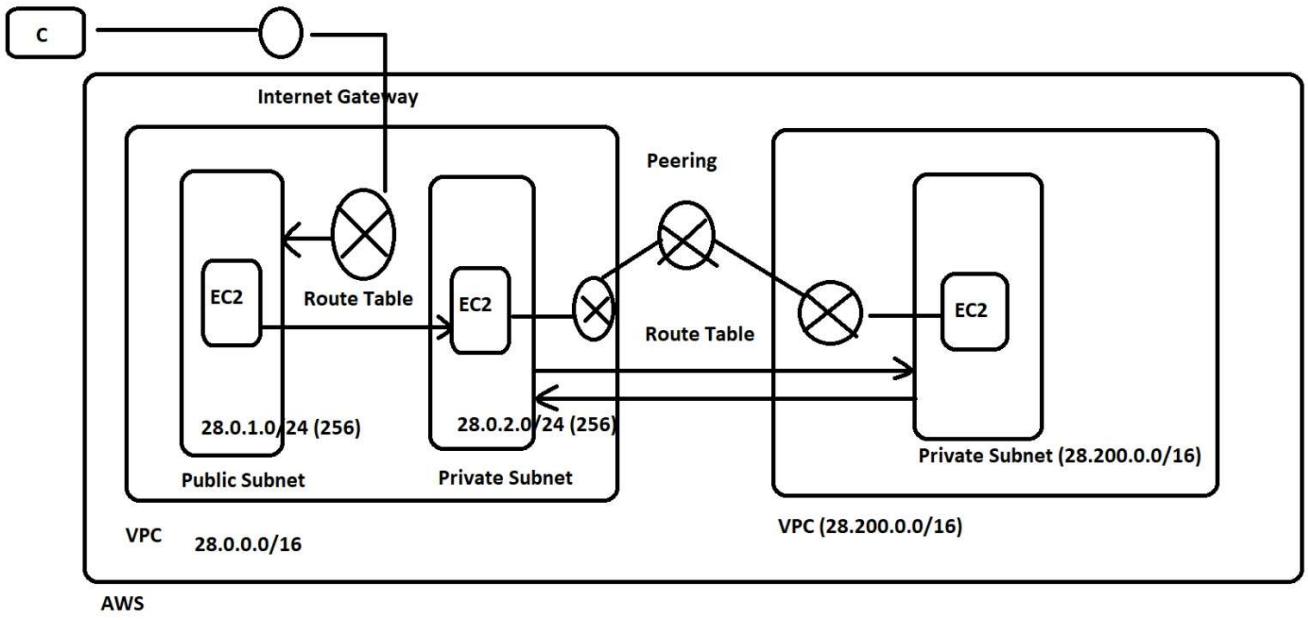
2.4 Architecture Diagram

Insert your architecture diagram here.

The diagram should show:

1. Your laptop connecting to the bastion host.
2. The bastion host in VPC-A's public subnet.
3. App-A in VPC-A's private subnet.
4. App-B in VPC-B's private subnet.
5. The VPC peering connection between VPC-A and VPC-B.
6. The private IP addresses and CIDR ranges.
7. The route from VPC-A to VPC-B and the return route.

Figure 1: AWS VPC Peering project architecture.



3. Implementation

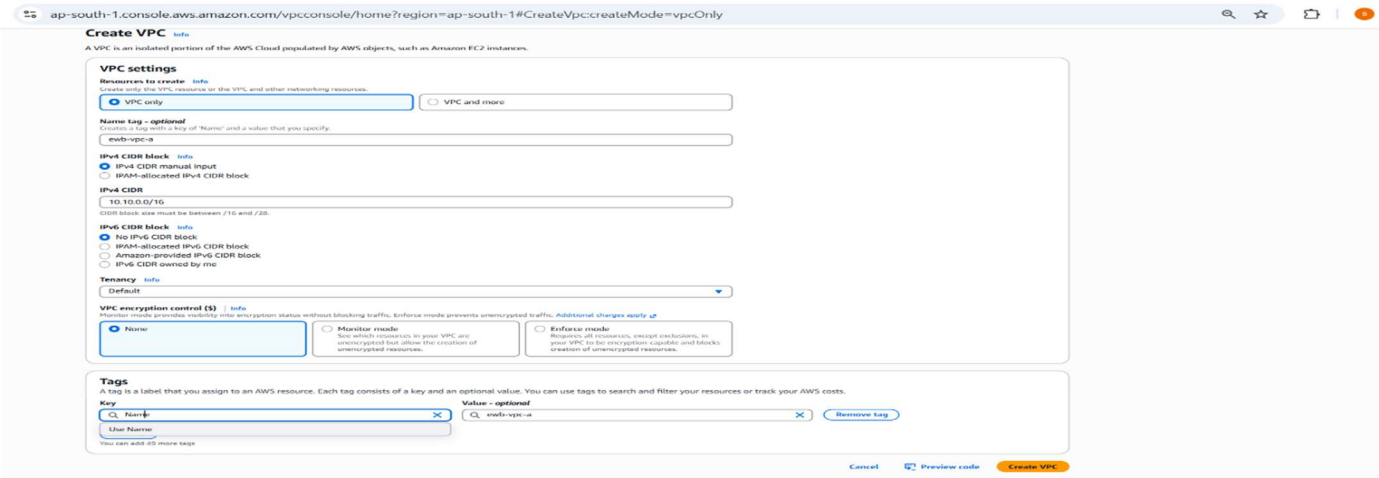
3.1 Create VPC-A

VPC-A was created with the name ewb-vpc-a and CIDR block 10.10.0.0/16.

Two subnets were configured:

- Public subnet: ewb-a-public
- Private subnet: ewb-a-private

Screenshot: Insert the VPC-A details and subnet configuration.



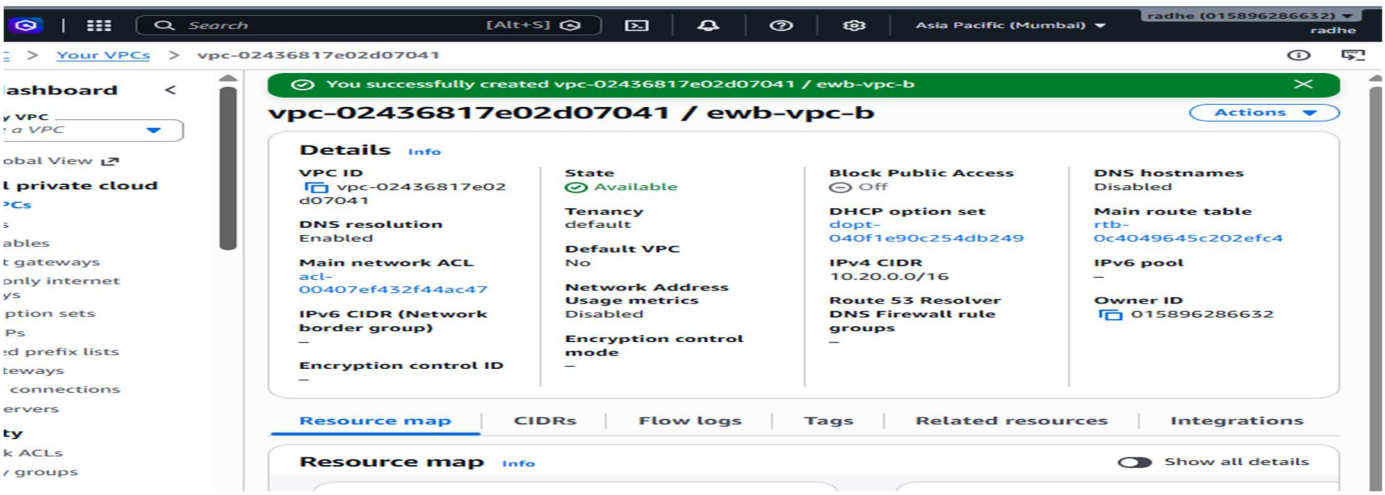
3.2 Create VPC-B

VPC-B was created with the name ewb-vpc-b and CIDR block 10.20.0.0/16.

The private subnet ewb-b-private was configured with CIDR 10.20.1.0/24.

VPC-B was designed without an Internet Gateway, keeping the application instance private.

Screenshot: Insert the VPC-B details and private subnet configuration.



3.3 Configure the Internet Gateway

The Internet Gateway ewb-igw-a was attached to VPC-A.

The public route table was configured with a default route to the Internet Gateway so the bastion host could be accessed using its public IPv4 address, subject to the relevant security rules.

Screenshot: Insert the Internet Gateway and public route table configuration.

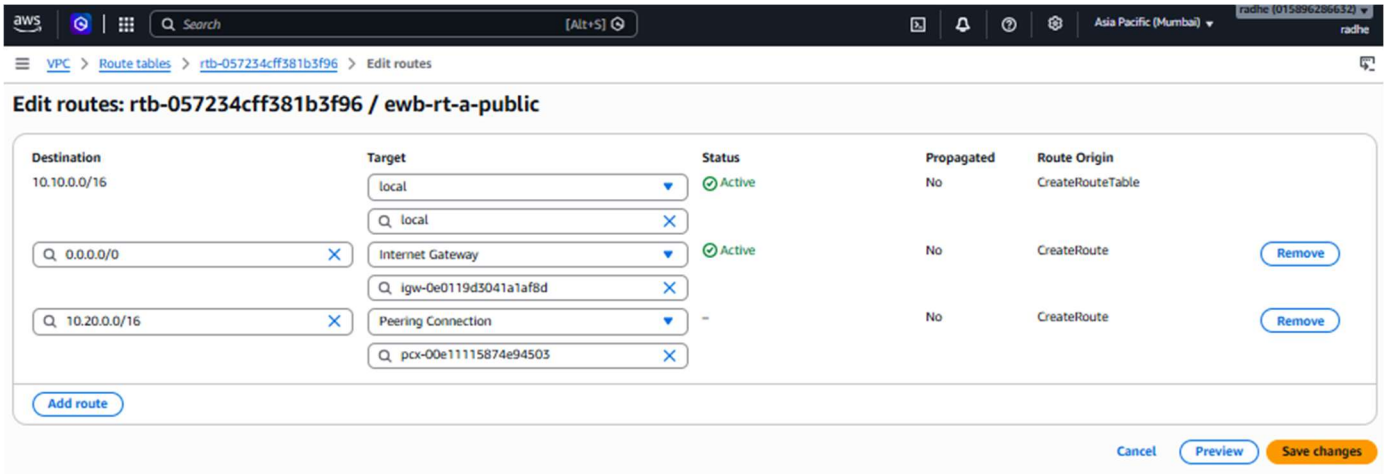
3.4 Establish VPC Peering

A peering connection named ewb-a-b-peer was created between VPC-A and VPC-B.

The connection was accepted and its status was verified as Active.

Peering connection ID: pcx-00e11115874e94503

Screenshot: Insert the peering connection details showing Active status and both VPCs.



3.5 Configure Route Tables

The private route table in VPC-A was configured to send traffic destined for VPC-B through the peering connection.

The private route table in VPC-B was configured to send traffic destined for VPC-A through the same peering connection.

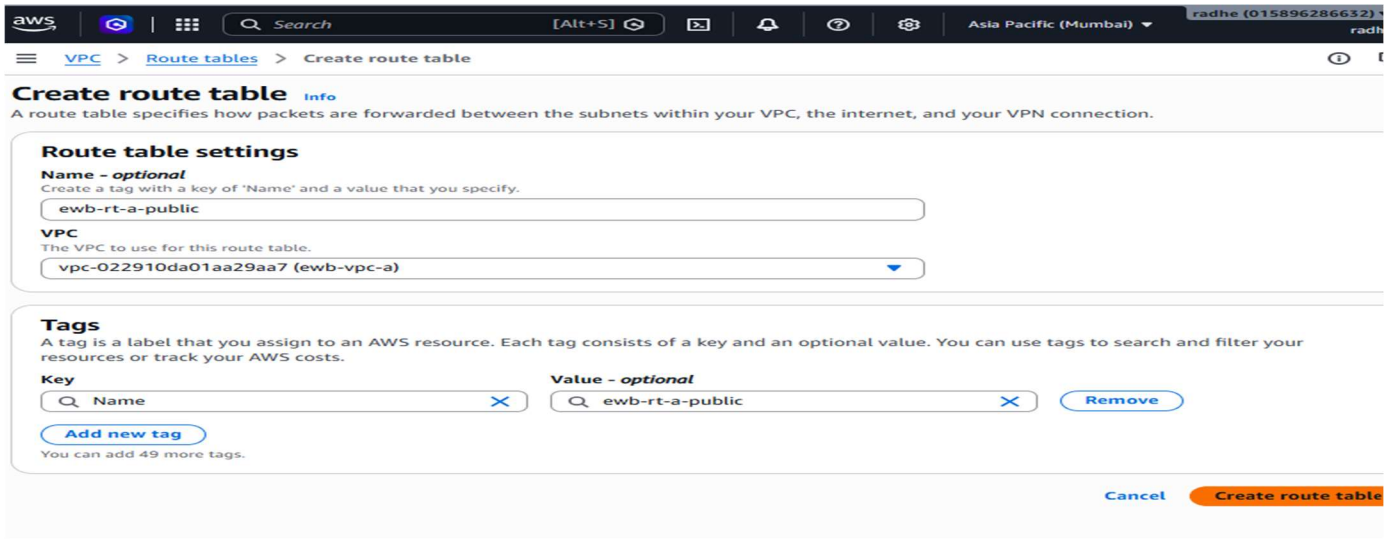
Route Table	Destination	Target
ewb-rt-a-private	10.20.0.0/16	VPC peering connection
ewb-rt-b-private	10.10.0.0/16	VPC peering connection

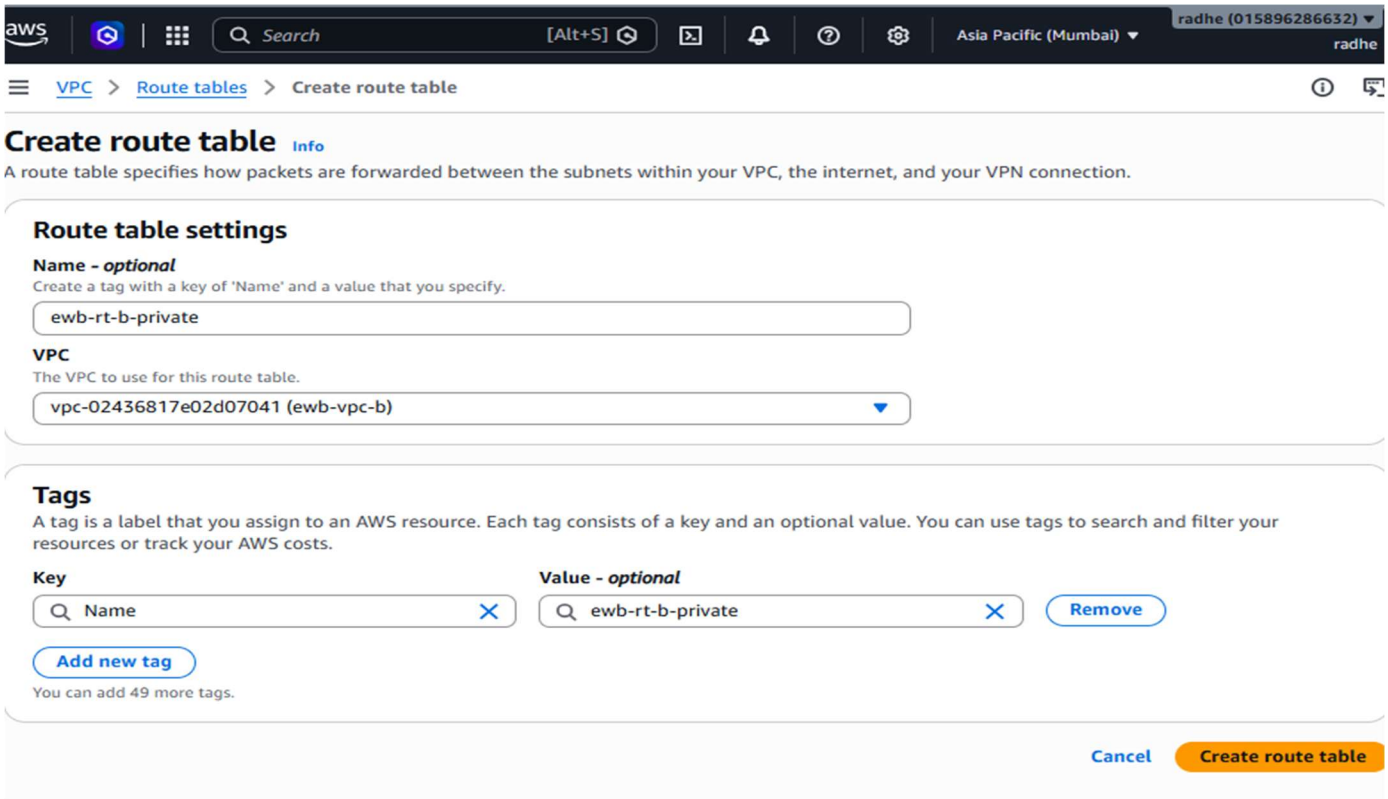
The public route table in VPC-A also requires a route to 10.20.0.0/16 through the peering connection so the bastion can reach App-B through the private network.

The existing local routes and the public default route to the Internet Gateway were retained.

Screenshots:

- VPC-A private route table
- VPC-B private route table
- VPC-A public route table





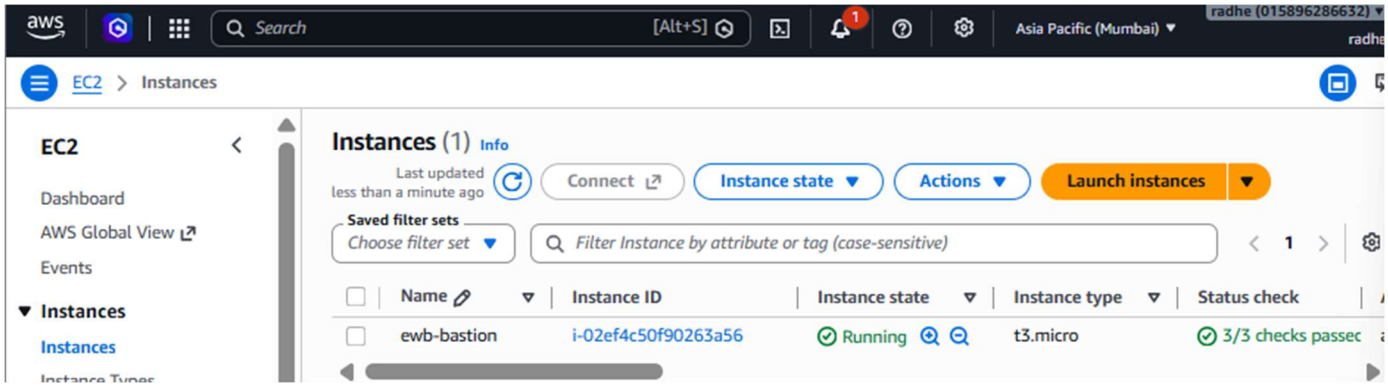
3.6 Launch the EC2 Instances

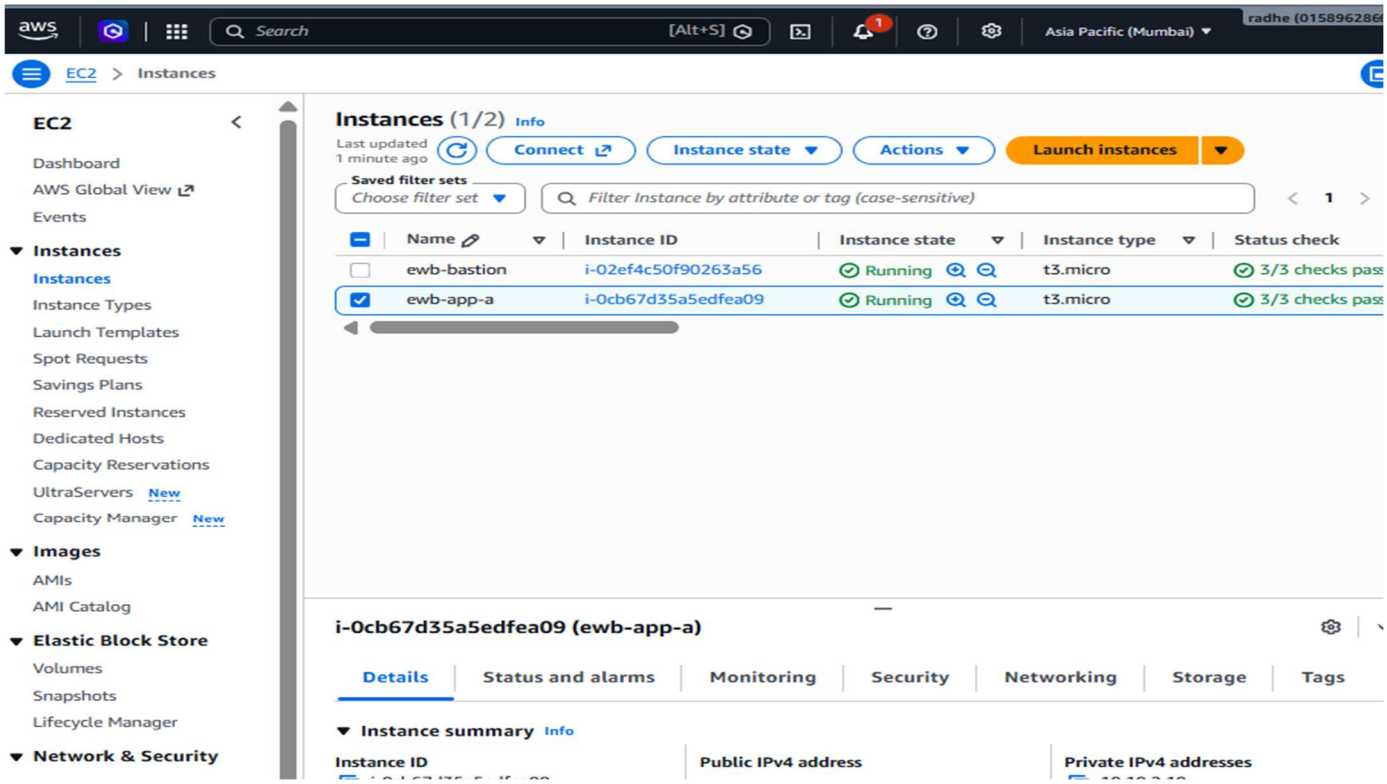
Three EC2 instances were used:

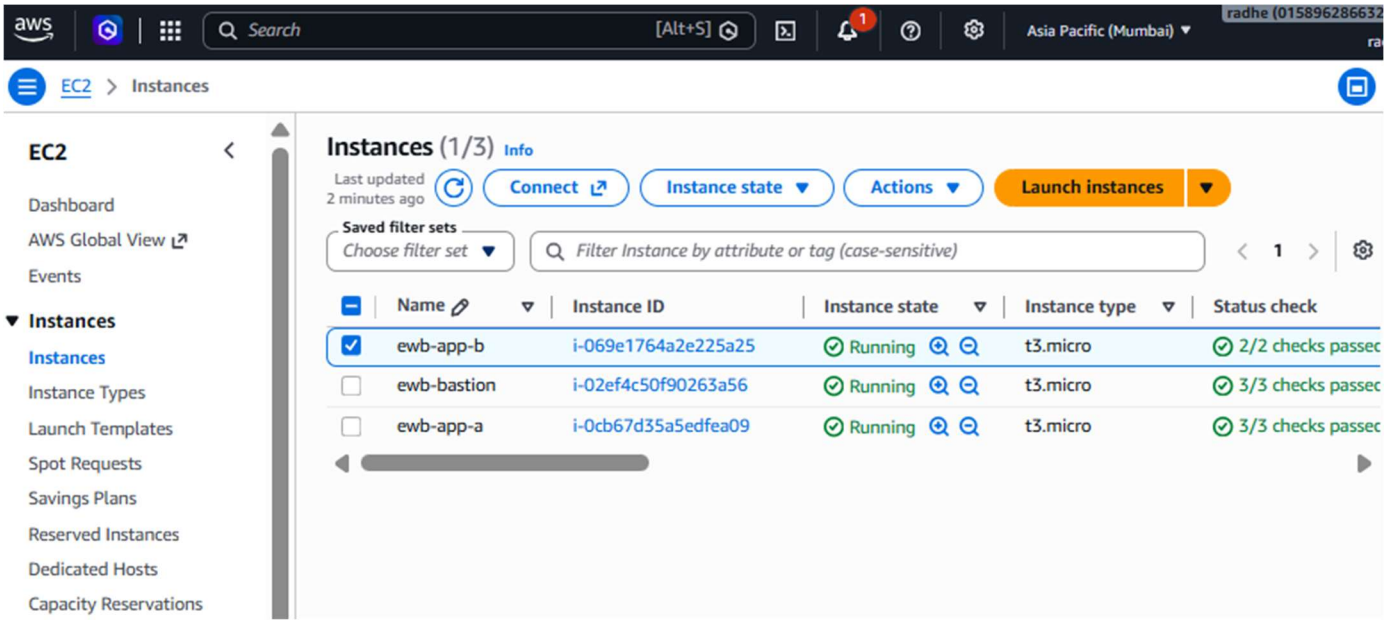
- ewb-bastion for administration.
- ewb-app-a as the private workload in VPC-A.
- ewb-app-b as the private workload in VPC-B.

App-A and App-B were configured without public IPv4 addresses.

Screenshots: Insert the instance details showing their names, VPCs, subnet associations, private IPs, and public IP status.







4. Security Configuration

Security groups were configured to permit only the required inbound traffic.

The bastion security group permits SSH access from the student's current public IPv4 address using a /32 source rule.

App-A and App-B permit SSH from the bastion's actual private IP address. ICMP Echo Request traffic is permitted between the two application servers using their individual private IP addresses.

The application servers do not require public IPv4 addresses for communication over VPC Peering.

5. Testing and Results

5.1 Baseline Test Before Peering

Run the following command on App-A before configuring peering and peer routes:

ping -c 4 10.20.1.10

Record the actual result and insert the terminal screenshot.

5.2 Forward Connectivity Test

Run the following command on App-A after configuring peering and the required routes:

ping -c 4 10.20.1.10

Observed result: Replies were received from App-B during the successful test.

Insert the terminal screenshot.

5.3 Reverse Connectivity Test

Run the following command on App-B:

ping -c 4 10.10.2.10

Observed result: Four packets were received, with 0% packet loss.

Insert the terminal screenshot.

5.4 Private SSH Test

From the local laptop, use the bastion host to access App-B through SSH ProxyJump.

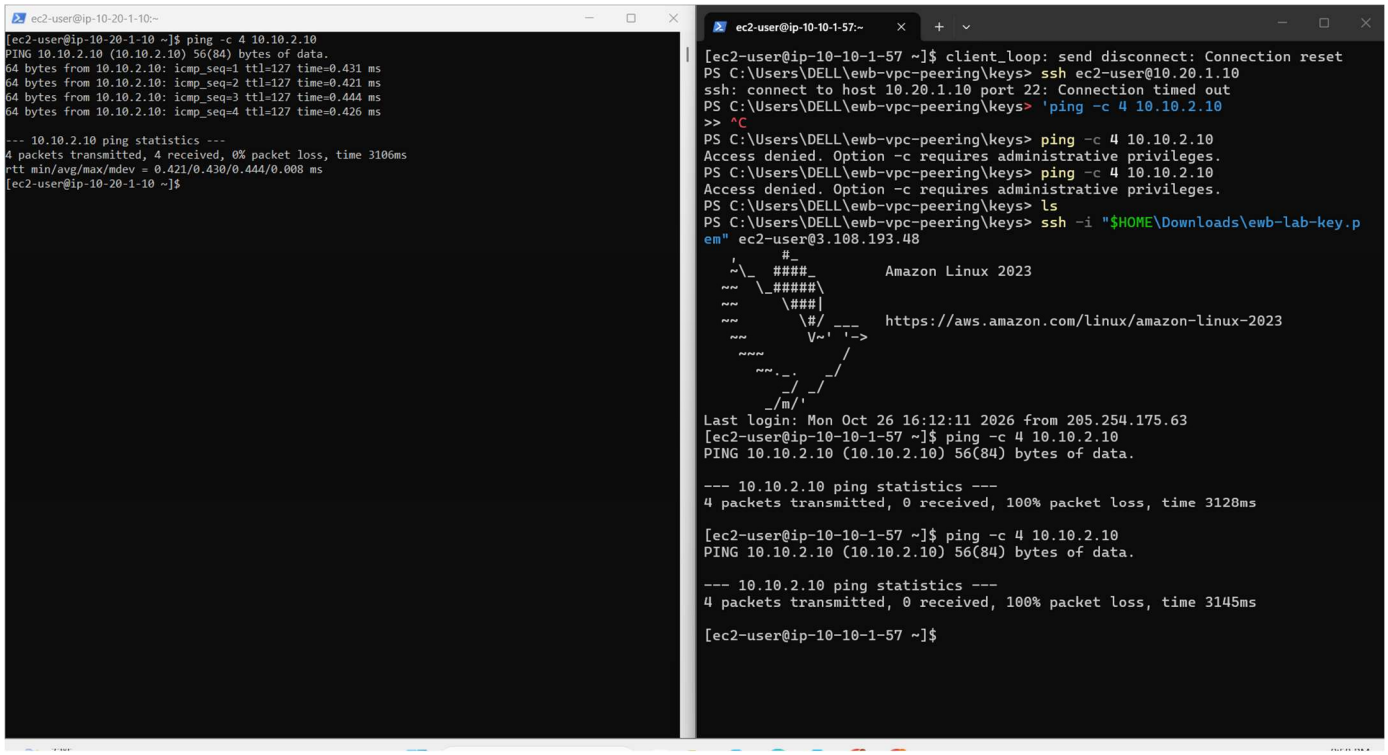
Record the command used and insert a screenshot showing the successful App-B shell session.

5.5 Controlled Failure and Recovery

Temporarily remove only the App-B inbound ICMP Echo Request rule that permits traffic from App-A.

Repeat the forward ping test from App-A and record the result.

Restore the exact rule and repeat the ping test to verify recovery.



6. Challenges and Troubleshooting

One challenge encountered during the project was attempting to connect directly from the local computer to a private EC2 instance.

The private IP address was not directly reachable from the public internet. The appropriate administration path is to connect through the bastion host and then access the private instance.

Other important troubleshooting checks include verifying the VPC peering status, route-table entries, subnet associations, security-group rules, private IP addresses, and SSH configuration.

7. Conclusion and Learning Outcomes

This project provided practical experience with AWS VPC networking, subnet configuration, route tables, VPC Peering, EC2 instances, and security groups.

Successful ping tests between App-A and App-B demonstrated private ICMP communication between the two VPCs.

The project also demonstrated why peering alone is insufficient: appropriate routes must exist in both directions, and security groups must permit the intended traffic.

The additional SSH, controlled-failure, recovery, and cleanup tests provide further evidence of network access control and troubleshooting skills.

8. Cleanup and Cost Management

After saving the required screenshots and project evidence, review all resources created specifically for this lab.

The cleanup process includes:

1. Terminating the lab EC2 instances.
2. Confirming that the associated EBS volumes have been deleted where appropriate.
3. Removing any unused Elastic IP addresses.
4. Deleting the peering connection and lab routes.
5. Removing the lab security groups, subnets, custom route tables, and Internet Gateway.
6. Deleting the two lab VPCs.
7. Verifying that no unintended chargeable lab resources remain.

Cleanup evidence: [Insert screenshots and record the actual cleanup status.]

9. References

- AWS VPC Peering documentation
- Amazon VPC User Guide

- Amazon EC2 documentation
- EWB Courses AWS VPC Peering Student Project Workbook